

Python – Formelsamling

Teknologi- og dataintegration · E25AB

1. Datatyper

Type	Betyder	Eksempel
str	Tekst (streng)	"Sara", "12"
int	Heltal	21, -5, 0
float	Decimaltal	3.14, 2.0
bool	Sandhedsværdi	True, False

`type(x)` fortæller dig, hvilken datatype `x` har.

2. Konvertering mellem datatyper

Funktion	Gør
<code>int(x)</code>	Laver x om til et heltal – kapper decimaler, runder ikke
<code>float(x)</code>	Laver x om til et decimaltal
<code>str(x)</code>	Laver x om til tekst

Pas på!

- `"12" + "8"` giver `"128"` (tekst sættes sammen), ikke `20`.
- `int(9.9)` giver `9` – ikke `10`. `int()` runder ikke, den fjerner bare alt efter kommaet.
- `"20" >= 18` giver **fejl** – man kan ikke sammenligne tekst (`str`) og tal (`int`) direkte.
- Tjek altid datatypen, hvis noget virker mærkeligt: `type(x)`.

3. Lister

En liste skrives med `[ ]` og indeholder elementer adskilt af komma. Det **første** element har indeks `0`.

```
1 frugter = ["æble", "banan", "pære"]
2 print(frugter[0]) # "æble" (det første)
3 print(frugter[-1]) # "pære" (det sidste)
```

Typiske liste-funktioner

Funktion	Gør
<code>len(liste)</code>	Antal elementer i listen
<code>liste[i]</code>	Elementet på plads i (indeks) – første plads er 0
<code>liste[-1]</code>	Det sidste element i listen
<code>liste.append(x)</code>	Tilføjer x sidst i listen
<code>for x in liste:</code>	Looper igennem hvert element i listen, ét ad gangen
<code>while i < len(liste):</code>	Looper med et tælleindeks i – bruges ofte sammen med <code>liste[i]</code>

4. Regneoperatorer

Tegn	Navn	Eksempel	Resultat
+	Plus	5 + 3	8
-	Minus	5 - 3	2
*	Gange	5 * 3	15
/	Division	9 / 4	2.25
//	Heltalsdivision (runder ned)	9 // 4	2
%	Modulo (resten ved division)	9 % 4	1

Modulo bruges tit til at tjekke lige/ulige tal: `tal % 2 == 0` er sandt, hvis `tal` er lige.

5. Sammenligningsoperatorer

Tegn	Betyder
<code>==</code>	Er lig med
<code>!=</code>	Er ikke lig med
<code>></code>	Større end
<code><</code>	Mindre end
<code>>=</code>	Større end eller lig med
<code><=</code>	Mindre end eller lig med

Resultatet af en sammenligning er altid `True` eller `False` (`bool`).

6. Boolske operatorer (and / or / not)

Operator	Betyder	Sand hvis...
<code>and</code>	Og	begge betingelser er sande
<code>or</code>	Eller	mindst én betingelse er sand
<code>not</code>	Ikke	vender resultatet om (True bliver False)

`True` tæller som `1` og `False` tæller som `0`, når de bruges sammen med `+` eller `-`:

```
1 pris = 100 + True # pris bliver 101
2 pris = 100 - False # pris bliver 100
```

- En flag-variabel (fx `fundet = False`) starter typisk som `False` og skiftes til `True`, når noget bestemt sker i koden.

7. If / elif / else

if / elif / else

Computeren tjekker betingelserne **oppefra og ned, én ad gangen**. Så snart én betingelse er sand:

- Den tilhørende gren køres.
- Resten af `elif/else` bliver **ikke** tjekket – heller ikke selvom de også ville have været sande.

```
1 karakter = 7
2
3 if karakter >= 10:      # False → videre
4     print("Fremragende")
5 elif karakter >= 7:     # True → STOP her
6     print("Godt")
7 elif karakter >= 4:     # springes over
8     print("Jævnt")
9 else:                  # springes over
10    print("Dumpet")
```

Output: `Godt`

Flere separate if-sætninger (uden elif)

Denne gang hører linje 3-6 sammen som et `if/else`-par – kun én af dem kan blive kørt. Linje 7-10 er derimod to **selvstændige** `if`-sætninger, som bliver tjekket helt uafhængigt af `if/else`-parret ovenfor (og af hinanden):

```

1 karakter = 7
2
3 if karakter >= 10:      # False → else
4     print("Fremragende")
5 else:                  # køres (if var falsk)
6     print("Ikke fremragende")
7 if karakter >= 4:      # selvstændig if
8     print("Jævnt")
9 if karakter < 4:       # False, springes over
10    print("Dumpet")

```

Output:

```

Ikke fremragende
Jævnt

```

Der bliver stadig printet **to** linjer: én fra if/else-parret (linje 3-6) og én fra den selvstændige if på linje 7. If/else-parret opfører sig som i afsnittet ovenfor – kun én gren køres – men de efterfølgende if-sætninger er ikke en del af det par, så de bliver tjekket, uanset hvad if/else endte med at gøre.

Sammenligning

	if / elif / else	if/else + selvstændige if
Hvor mange betingelser bliver tjekket?	Stopper ved den første sande	Hvert if/elif/else- par stopper for sig – men de næste, selvstændige if-sætninger bliver altid tjekket
Hvor mange grene kan blive kørt?	Højst én i alt	Højst én pr. if/elif/else-par , men flere samlet set

8. Løkker (loops)

for-loop med `range(start, stop, step)`

Tæller fra `start` op til (men **ikke med**) `stop`, i spring på `step` ad gangen. `step` er `1`, hvis den ikke skrives.

Kode	Værdi af t
for t in range(5):	0 1 2 3 4
for t in range(1, 5):	1 2 3 4
for t in range(2, 11, 2):	2 4 6 8 10

while-loop

Kører, **så længe** betingelsen er sand. Husk at ændre variabelen inde i loopet, ellers kører det for evigt.

```

1 tal = 5
2 while tal > 0:
3     print(tal)
4     tal = tal - 1
5 # tal: 5, 4, 3, 2, 1

```

break

Stopper loopet med det samme, uanset betingelsen. I et nested loop (loop i loop) stopper `break` kun det **inderste** loop – det ydre loop fortsætter.

Loop i loop (nested loop)

For hver værdi i det ydre loop kører **hele** det indre loop igennem, før det ydre skifter til næste værdi.

```

1 for i in range(1, 3):
2     for j in range(1, 3):
3         print(i, j)
4 # (1,1) (1,2) (2,1) (2,2)

```

9. Strings (tekst) som en liste af bogstaver

Et for-loop kan gå igennem bogstaverne i en tekst, ét ad gangen.

`len(tekst)` tæller antal bogstaver.

```

1 for bogstav in "kode":
2     print(bogstav)
3 # k
4 # o
5 # d
6 # e

```

10. Sådan arbejder du med kode på papir

- Koden køres **altid** oppefra og ned, linje for linje.
- En variabel husker sin værdi, indtil en ny linje ændrer den.
- Lav en lille tabel med én kolonne pr. variabel, og opdater den for hver omgang i et loop.
- Ved nested loops: brug to kolonner (fx `i` og `j`), og husk at det indre loop kører helt færdigt, før det ydre tæller videre.